

15.5: Linked Data Structures

[This section corresponds to K&R Sec. 6.5]

One reason that pointers to structures are useful and common is that they can be used to build linked data structures, in which a structure contains a pointer to another instance of the same structure (or perhaps a different structure). The simplest example is a singly-linked list, which we might declare like this:

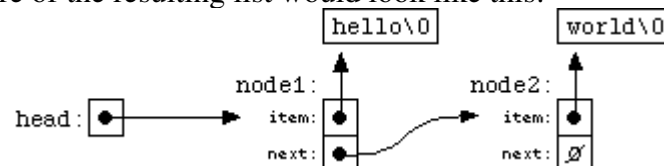
```
struct listnode
{
    char *item;
    struct listnode *next;
};
```

This structure describes one *node* in a list; a list may consist of many nodes, one for each item in the list. Here, each item in the list (the field we've named *item*) will be a string, represented (i.e. pointed to) by a `char *`. Each node in the list is linked to its successor by its *next* field, which is a pointer to another `struct listnode`. (The compiler is perfectly happy to place a pointer to a structure inside that very same structure; it would only complain if you tried to stuff an entire `struct listnode` into a `struct listnode`, which would tend to make the `struct listnode` explode.) We'll use a null pointer as the *next* field of the last node in the list, since by definition a null pointer doesn't point anywhere.

We could set up a tiny list with these declarations:

```
struct listnode node2 = {"world", NULL};
struct listnode node1 = {"hello", &node2};
struct listnode *head = &node1;
```

A box-and-arrows picture of the resulting list would look like this:



The list has two nodes, allocated by the compiler in response to the first two declarations we gave. We've also allocated a pointer variable, *head*, which points at the "head" of the list.

Once we've built a list, we'll naturally want to do things with it. One of the simplest operations is to print the list back out. The code to do so is very simple. We declare another list pointer *lp* and then cause it to step over each node in the list, in turn:

```
struct listnode *lp;
for(lp = head; lp != NULL; lp = lp->next)
    printf("%s\n", lp->item);
```

This `for` loop deserves some attention, especially if you haven't seen one like it before. Many `for` loops step an `int` variable (often called *i*) through a series of integer values. However, the three controlling expressions of a `for` loop are *not* limited to that pattern; you may in fact use any expressions at all, although it's best if they conform to the expected initialize;test;increment pattern. The list-printing loop above certainly does: the expression `lp = head` initializes *lp* to point to the head of the loop; the expression `lp != NULL` tests whether *lp* still points to a real node (or whether it has reached the null pointer which marks the end of the list); and the expression `lp = lp->next` sets

`lp` to point to the next node in the list, one past where it did before.

The two-element list above is pretty useless; its only worth is as a first example. The real power of linked lists (and other linked data structures) is that they can grow on demand, in response to the data that your program finds itself working with. For a linked list to grow on demand, however, we'll have to allocate its nodes dynamically, because we won't know in advance how many of them we'll need. (In the first example, we had two static nodes, because we knew in advance, at compile time, that our list would have two elements. But that static allocation won't do for a dynamic list. How would we know how many `struct listnode` variables to allocate?)

The general solution, of course, is to call `malloc`. Here is a scrap of code which inserts a new word at the head of a list:

```
#include <stdio.h>                /* for fprintf, stderr */
#include <stdlib.h>                /* for malloc */

char *newword = "test";
struct listnode *newnode = malloc(sizeof(struct listnode));
if(newnode == NULL)
{
    fprintf(stderr, "out of memory\n");
    exit(1);
}

newnode->item = newword;
newnode->next = head;
head = newnode;
```

The expression `sizeof(struct listnode)` in the call to `malloc` asks the compiler to compute the number of bytes required to store one `struct listnode`, and that's exactly how many bytes of memory we ask for from `malloc`. Make sure you see how the last two lines work to splice the new node in at the head of the list, by making the new node's `next` pointer point at the old head of the list, and then resetting the head of the list to be the new node. (Another word for a list where we always add new items at the beginning is a *stack*.)

Naturally, we'd like to encapsulate this operation of prepending an item to a list as a function. Doing so is just a little bit tricky, because the list's head pointer is modified every time. There are several ways to achieve this modification; the way we'll do it is to have our list-prepending function return a pointer to the new head of the list. Here is the function:

```
#include <stdio.h>                /* for fprintf, stderr */
#include <stdlib.h>                /* for malloc, exit */
#include <string.h>                /* for strlen, strcpy */

struct listnode *prepend(char *newword, struct listnode *oldhead)
{
    struct listnode *newnode = malloc(sizeof(struct listnode));
    if(newnode == NULL)
    {
        fprintf(stderr, "out of memory\n");
        exit(1);
    }

    newnode->item = malloc(strlen(newword) + 1);
    if(newnode->item == NULL)
    {
        fprintf(stderr, "out of memory\n");
        exit(1);
    }
}
```

```
strcpy(newnode->item, newword);

newnode->next = oldhead;
return newnode;
}
```

Since we want this to be a general-purpose function, we also allocate new space for the new string (word, item) being stored. Otherwise, we'd be depending on the caller to arrange that the pointer to the new string remain valid for as long as the list was in use. As we'll see, that's not always a safe assumption. By allocating our own memory, which ``belongs" to the list, we ensure that the list isn't dependent on the caller in this way. (Notice, too, that the number of bytes we ask for is `strlen(newword) + 1`.)

(As an aside, it's a mild blemish on the above code that it contains two identical calls to `fprintf`, complaining about two separate potential failures in two calls to `malloc`. It's quite possible to combine these two cases, and many C programmers prefer to do so, although the expression may be a bit scary-looking at first:

```
struct listnode *newnode;
if((newnode = malloc(sizeof(struct listnode))) == NULL ||
    (newnode->item = malloc(strlen(newword) + 1)) == NULL)
{
    fprintf(stderr, "out of memory\n");
    exit(1);
}
```

How does this work? First it calls `malloc(sizeof(struct listnode))`, and assigns the result to `newnode`. Then it calls `malloc(strlen(newword) + 1)`, and assigns the result to `newnode->item`. This code relies on two special guarantees of C's `||` operator, namely that it always evaluates its left-hand side first, and that if the left-hand side evaluates as true, it doesn't bother to evaluate the right-hand side, because once the left-hand side is true, the final result is definitely going to be true. Therefore, we're guaranteed that we'll allocate space for `newnode` to point to before we try to fill in `newnode->item`, but if the first call to `malloc` fails, we won't call `malloc` a second time or try to fill in `newnode->item` at all. These guarantees--of left-to-right evaluation, and of skipping the evaluation of the right-hand side if the left-hand side determines the result--are unique to the `||` and `&&` operators. It's perfectly acceptable to rely on these guarantees when using `||` and `&&`, but don't assume that other operators will operate deterministically from left to right, because most of them don't.)

Now that we have our `prepend` function, we can build a list by calling it several times in succession:

```
struct listnode *head = NULL;
head = prepend("world", head);
head = prepend("hello", head);
```

This code builds essentially the same list as our first, static example. Notice how we initialize the list head pointer with a null pointer, which is synonymous with an empty list. (Notice also that the code we wrote up above, for printing a list, would also deal correctly with an empty list.)

Using static calls to `prepend` is hardly more interesting than building a static link by hand. To make things truly interesting, let's read words or strings typed by the user (or redirected from a file), and prepend those to a list. The code is not hard:

```
#define MAXLINE 200

char line[MAXLINE];
```

```
struct listnode *head = NULL;
struct listnode *lp;

while(getline(line, MAXLINE) != EOF)
    head = prepend(line, head);

for(lp = head; lp != NULL; lp = lp->next)
    printf("%s\n", lp->item);
```

(`getline` is the line-reading function we've been using. If you don't have a copy handy, you can use the `fgets` function from the standard library.)

If you type in this code and run it, you will find that it prints lines back out in the reverse order that you typed them. (In doing so, of course, it slurps all the lines into memory, so you might run out of memory if you tried to use this technique for reversing all the lines in a huge file.) Notice that when we call `prepend` in this way, it *is* important that `prepend` allocate memory for, and stash away, each string. Can you see what would happen if `prepend` did not, that is, if it simply said `newnode->item = newword`?

Linked lists are only the simplest example of linked data structures. There are also queues, doubly-linked lists, trees, and circular lists. We'll see more concrete examples of linked structures in the adventure game example. The set of objects sitting in a room (or held by the player) will be represented by a linked list of objects, and the rooms will be linked to each other to indicate the passages between rooms.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1996-1999 // [mail feedback](#)